Relational Operators in Java

Relational operators are used to compare two values, variables, or expressions. These operators return a boolean value (`true` or `false`) based on the relationship between the operands.

---

1. List of Relational Operators

Operator	Description	Example
`==`	Equal to	`a == b`
`!=`	Not equal to	`a != b`
`>`	Greater than	`a > b`
`<`	Less than	`a < b`
`>=`	Greater than or equal to	`a >= b`
`<=`	Less than or equal to	`a <= b`

---

2. Description of Relational Operators

1. Equal to (`==`)

- Checks if two values are equal.
- Returns `true` if they are equal, otherwise `false`.

Example:

```
public class EqualToExample {
    public static void main(String[] args) {
        int a = 10, b = 10;
        System.out.println("Is a equal to b? " + (a == b)); // true
    }
}
```

---

2. Not Equal to (`!=`)

- Checks if two values are not equal.
- Returns `true` if they are not equal, otherwise `false`.

Example:

```
public class NotEqualToExample {
    public static void main(String[] args) {
        int a = 10, b = 20;
        System.out.println("Is a not equal to b? " + (a != b)); // true
    }
}
```

---

3. Greater than (`>`)

- Checks if the left operand is greater than the right operand.
- Returns `true` if it is, otherwise `false`.

Example:

```
public class GreaterThanExample {
    public static void main(String[] args) {
        int a = 15, b = 10;
        System.out.println("Is a greater than b? " + (a > b)); // true
    }
}
```

---

4. Less than (`<`)

- Checks if the left operand is less than the right operand.
- Returns `true` if it is, otherwise `false`.

Example:

```
public class LessThanExample {
    public static void main(String[] args) {
        int a = 5, b = 10;
        System.out.println("Is a less than b? " + (a < b)); // true
    }
}
```

---

5. Greater than or Equal to (`>=`)

- Checks if the left operand is greater than or equal to the right operand.
- Returns `true` if it is, otherwise `false`.

Example:

```
public class GreaterThanOrEqualExample {
    public static void main(String[] args) {
        int a = 20, b = 20;
        System.out.println("Is a greater than or equal to b? " + (a >= b));
// true
    }
}
```

---

6. Less than or Equal to (`<=`)

- Checks if the left operand is less than or equal to the right operand.
- Returns `true` if it is, otherwise `false`.

Example:

```
public class LessThanOrEqualExample {
    public static void main(String[] args) {
        int a = 10, b = 15;
        System.out.println("Is a less than or equal to b? " + (a <= b)); //
true
    }
}
```

---

3. Key Points

1. Relational operators are often used in conditional statements like `if`, `while`, and `for` loops.
2. Operands must be comparable types (`int`, `float`, `char`, etc.).
3. When comparing objects (e.g., `String`), use `equals()` instead of `==` to check for content equality.

---

4. Comprehensive Example

```
public class RelationalOperatorsDemo {
    public static void main(String[] args) {
        int a = 10, b = 20;

        System.out.println("a == b: " + (a == b)); // false
        System.out.println("a != b: " + (a != b)); // true
        System.out.println("a > b: " + (a > b));   // false
        System.out.println("a < b: " + (a < b));   // true
        System.out.println("a >= b: " + (a >= b)); // false
        System.out.println("a <= b: " + (a <= b)); // true

        // Comparing floating-point numbers
        double x = 5.5, y = 5.5;
        System.out.println("x == y: " + (x == y)); // true

        // Comparing characters
        char c1 = 'A', c2 = 'B';
        System.out.println("c1 < c2: " + (c1 < c2)); // true (ASCII
comparison)

        // String comparison
        String str1 = "Hello", str2 = "Hello";
        System.out.println("str1 == str2: " + (str1 == str2)); // true
(because of String interning)
        System.out.println("str1.equals(str2): " + str1.equals(str2)); //
true
    }
}
```

---

5. Output

```
a == b: false
a != b: true
a > b: false
a < b: true
a >= b: false
a <= b: true
x == y: true
c1 < c2: true
str1 == str2: true
str1.equals(str2): true
```

6. Common Mistakes

1. Using `==` for object comparison instead of `equals()`.
    - Example:

```
String s1 = new String("Hello");
String s2 = new String("Hello");
System.out.println(s1 == s2);         // false (compares references)
System.out.println(s1.equals(s2));  // true (compares values)
```

2. Mixing types (e.g., comparing `int` with `String`) causes a compilation error.